

LAB 01

System Performance Evaluation under Idle and Heavy Load Conditions

System Configuration and Performance Evaluation Lab
Course No: 0613-310

Ekramul Haque Amin
Lecturer, AI and Data Science
Dhaka International University (DIU)

Today's Objectives

1. Measure system performance in the idle condition
2. Measure system performance under heavy load
3. Compare the results and explain the differences

Technique used today: Measurement on a real computer

Why Evaluate Performance?

- Compare two systems or configurations before buying
- Find the bottleneck: CPU, memory or disk
- Tune a system so it runs faster
- Plan capacity for future users and workload

Goal: get the highest performance for a given cost

Three Evaluation Techniques

Criterion	Analytical Modeling	Simulation	Measurement
Stage	Any	Any	After prototype
Time required	Small	Medium	Varies
Tools	Analysts	Programming languages	Instrumentation
Accuracy	Low	Moderate	Varies
Cost	Small	Medium	High
Persuasiveness	Low	Medium	High

Today we use Measurement

Performance Metrics We Will Measure

Metric	What it tells us	Unit	Better
CPU Utilization	How busy the processor is	%	Lower = more free
RAM Utilization	How much memory is in use	%	Lower = more free
Benchmark Time	Time to finish a fixed job	seconds	Lower
Throughput	Work done per second	numbers/sec	Higher
Response Time	Time for a quick task	seconds	Lower
Disk Speed	Data written/read per second	MB/s	Higher

Idle vs Heavy Load

Idle

- Only the OS and our program run
- CPU is mostly free
- Our program gets the CPU at once

Heavy Load

- Many programs compete for CPU, RAM and disk
- OS shares CPU in small time slices
- Our program waits in a queue

Same program + more competition = slower results

Two Ways to Measure Time

Wall Time

- Real clock time from start to end
- Includes waiting for the CPU
- `time.perf_counter()`

CPU Time

- Time the CPU actually worked on our program
- Does not include waiting
- `time.process_time()`

Under load, wall time grows much more than CPU time

Tools Required

- Python 3
- psutil library – reads CPU, memory, disk and network usage
- performance_evaluation.py – the evaluation script

Install and run:

```
pip install psutil  
python performance_evaluation.py
```

What the Script Measures

Step	Function	Measures
1	<code>measure_idle_utilization()</code>	Average CPU % and RAM % over 5 seconds
2	<code>cpu_benchmark()</code>	Time to count primes below 50,000 + throughput
3	<code>response_time_test()</code>	Time for <code>sum(range(1,000,000))</code>
4	<code>disk_speed_test()</code>	Write and read a 100 MB file → MB/s
5	<code>current_disk_usage()</code> , <code>network_stats()</code>	Disk used %, data sent and received

Code: CPU and RAM Utilization

```
def measure_idle_utilization(duration=5):  
    cpu_samples = []  
    mem_samples = []  
    for _ in range(duration):  
        cpu_samples.append(psutil.cpu_percent(interval=1))  
        mem_samples.append(psutil.virtual_memory().percent)  
    return {  
        "Average CPU Utilization (%)": round(mean(cpu_samples), 2),  
        "Average RAM Utilization (%)": round(mean(mem_samples), 2)  
    }
```

cpu_percent(interval=1) watches the CPU for 1 second; 5 samples are averaged

Code: CPU Benchmark (Prime Counting)

```
start = time.perf_counter()
count = 0
for num in range(2, limit):
    is_prime = True
    root = int(math.sqrt(num)) + 1
    for i in range(2, root):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        count += 1
elapsed = time.perf_counter() - start
throughput = limit / elapsed
```

Fixed amount of work, so only the time changes

Throughput = numbers checked per second

Code: Response Time and Disk Speed

```
start = time.perf_counter()
total = sum(range(1000000))          # quick task
response_time = time.perf_counter() - start

data = os.urandom(1024 * 1024)       # 1 MB block
start_write = time.perf_counter()
with open(file_path, "wb") as f:
    for _ in range(file_size_mb):    # 100 MB
        f.write(data)
write_speed = file_size_mb / (time.perf_counter() - start_write)
```

Read speed can look very high because the OS caches the file in RAM

Step 1 – Idle Measurement

1. Close all unnecessary applications
2. Wait 1–2 minutes for the system to settle
3. Run: `python performance_evaluation.py`
4. Do not touch the computer while it runs
5. Screenshot the output and record CPU %, RAM %, benchmark time, response time and disk speed

Step 2 – Heavy Load Measurement

- 1. Open 15–20 Chrome tabs with YouTube videos playing**
- 2. Open VS Code and other heavy applications**
- 3. Wait until CPU usage stays high (check Task Manager)**
- 4. Run the same script again – no code changes**
- 5. Screenshot the output and record the same metrics**

Step 3 – Compare the Results

Metric	Idle	Load	Difference
CPU Usage (%)			
RAM Usage (%)			
Benchmark Time (s)			
Response Time (s)			
Disk Write Speed (MB/s)			
Disk Read Speed (MB/s)			

Difference = Load – Idle · % Change = (Load – Idle) ÷ Idle × 100

What to Watch For

Metric	Under heavy load	Reason to check
CPU Usage	Increases	Other programs use the CPU
RAM Usage	Increases	Every tab and app holds memory
Benchmark Time	Increases	Program waits for CPU time
Response Time	Increases	Quick task is queued behind others
Disk Speed	May drop or stay similar	Depends on disk use and OS cache

Analysis Questions

1. Why does CPU usage increase under load?
2. Why does response time become slower?
3. Which metric is most affected by load?
4. Is disk performance affected? Why or why not?

Answer these in your lab report, using your own numbers

Lab Report 01 – What to Submit

Sections

- Lab Title
- Objective
- Tools Required
- Procedure
- Results table
- Analysis
- Conclusion

Attach

- Screenshots of both runs
- Your system specs from the output
- Submit by: [date]

Tips for Fair Measurement

- Use the same computer and script for both runs
- Keep the laptop plugged in – battery saver slows the CPU
- Run each condition 3 times and take the average
- Do not move the mouse or type during a run
- Record your system specs with the results

Key Takeaway

Performance is not fixed. It depends on the workload running alongside your program.

Next lab: Benchmarking with the Sieve of Eratosthenes – CPU time vs wall time